

Password Validator

Important AI Statement: Students are strictly prohibited from using AI tools (ChatGPT, Copilot, or any other code-generating software) to complete this assignment. All code must be written manually by the student. Any detected AI-generated code will result in a failing grade for that assignment.

- **Requirement:** Submit only your own original code. Do NOT copy, paste, or modify code generated by AI. You may refer to textbooks, class notes, or personal experimentation only.
- **Honor Statement:** By submitting this assignment, you affirm that all code is your own work and was not created using AI code generation tools.

Penalty Statement: Violations of this policy will be treated as academic dishonesty, subject to the institution's academic integrity rules and 0 for this assignment.

In Python 1 and during the first few weeks of Python 2 you learned the Python topics of variables, conversions, arithmetic statements, logical constructs, looping constructs, functions, lists and string methods. This final project is the culmination of all those topics for you to demonstrate all that you have learned during the past 15 weeks.

Note: You can **NOT USE** Python's Regular Expression Operations known better as regex. You can only use what you learned in this class:

- variables
- conversions
- arithmetic statements
- logical constructs
- looping constructs
- functions
- lists
- string methods

Use Python String functions to build a program to do the following to perform Password Validation. Make sure you use the examples Professor Candido has provided and use the **Python String Functions** document found in Week 2.

1. Put s prefixes in front of variables that contain string values and i prefix for variables that contain integer numbers and f for floating numbers. Add your initials such as BLC at the end of each variable (not constants). For example: sPasswordBLC.
2. Prompt the user for their First and Last name and store the input into a variable called sName???. Replace ??? with your initials.
3. Code a loop that will keep asking for a password UNTIL a valid password is entered.
4. Prompt the user for their desired password and store their response in a variable called sPassword???. Replace ??? with your initials.
5. Extract out the first initial from the first name and the first initial of the last name from the variable sName and put in a new variable called sInitials???. Replace ??? with your initials. For example if Brian Candido is in the sName variable then the sInitials should contain BC. You can assume the user will enter 1 first name and 1 last name.
6. Check to make sure the sPassword length is between 8 and 12 characters. If the password is not within the required length write out the message: *Password must be between 8 and 12 characters.*
7. Check to make sure the sPassword starts does not start with Pass or pass. If not write out the *Password can't start with Pass.*
8. Check to make sure the sPassword contains at least 1 uppercase letter A through Z. If not write out the message: *Password must contain at least 1 uppercase letter.*
9. Check to make sure the sPassword contains at least 1 lowercase letter a through z. If not write out the message: *Password must contain at least 1 lowercase letter.*
10. Check to make sure the sPassword contains at least 1 number between 0 and 9. If not write out the message: *Password must contain at least 1 number.*

11. Check to make sure the `sPassword` contains at least 1 of these special characters: `!@#$%^`. If not write out the message: *Password must contain at least 1 of these special characters: !@#\$%^*
12. Check to make sure the `sPassword` does not contain the value of `sInitials` within the string. For example `sPassword` cannot contain BC or bc or any variation of BC so you will need to convert to either lowercase or uppercase than compare using Python string functions. If not write out the message: *Password must not contain user initials.*
13. No character can be present more than once. Write code to process each character in the password and keep track of how many occurrences are present. If any character (either uppercase or lowercase versions) is in the password more than 1 output the character and how many occurrences and print out the message: *These characters appear more than once:*
14. If `sPassword` passes all the above checks output this message: *Password is valid and OK to use.* And exit the loop.
15. Make sure your code has a `main()` function. It is your optional choice if you want additional functions.
16. This assignment is for learning purposes you cannot use AI/ChatGPT to create the code.
17. Use comments to document your code. At the top of your code place a Comment Heading Block that has these elements: Your Name, Code Name and you must answer these reflection questions:
 - Share what did you liked about this assignment?
 - Share what you struggled with?
 - How did you write your code to be efficient and reduce redundancy?
 - Share your top 2 things you learned on this assignment

```
# Name: YourName
# Assignment: Password Validator
# Reflection: Share what you liked about this assignment?
#           Share what you struggled with?
#           How did you write your code to be efficient and reduce redundancy?
#           Share exactly 2 things you learned on this assignment:
#           1.
#           2.
```

Sample Output

Password is not valid:

```
Enter full name such as John Smith: Brian Candido
Enter new password: BC12345
Password must be between 8 and 12 characters
Password must contain at least 1 lowercase letter
Password must contain at least 1 of these special characters: !@#$%^
Password must not contain user initials.

Enter new password:
```

Password is not valid:

```
Enter new password: abc!123
Password must be between 8 and 12 characters
Password must contain at least 1 uppercase letter
Password must not contain user initials.

Enter new password:
```

Password is not valid:

```
Enter new password: PasSW0rd
Password can't start with Pass.
Password must contain at least 1 of these special characters: ! @ # $ %
These characters appear more than once:
s: 2 times
Enter new password:
```

Password is not valid:

```
Enter new password: Mkc!C06182006
Password must be between 8 and 12 characters
These characters appear more than once:
c: 2 times
0: 3 times
6: 2 times
Enter new password:
```

Password is valid:

```
Enter new password: Mkc!0618
Password is valid and OK to use.
```

Grading Rubric

Criteria	Meets (100%)	Somewhat (50%)	Not Present (0%)
Password validations steps 5 through 11 are coded using Python String functions. 55 points	All String validation requirements stated in the instructions Steps 3 through 9 are present, functioning and coded efficiently.	Some of String validation requirements stated in the instructions Steps 3 through 9 are attempted and mostly functioning.	String validation requirements stated in the instructions Steps 3 through 9 are not implemented.
Code to make sure the Password does not contain the user's initials (step 12). 10 points	The code to detect the user's initials is present and coded correctly.	The code to detect the user's initials is attempted and partially works.	The code to detect the user's initials is not present.
Code to check for repeating characters (step 13). 15 points	The code to detect repeating characters is present and coded correctly.	The code to detect repeating characters is attempted and partially works.	The code to detect repeating characters is not present.
Comment Block Reflections, Comments and	Comment Heading Block is present and	Either Comment Heading Block/Comments present	No Comment Heading Block/Comments present

Variable Prefixing and Suffixing 20 points	reflection responses are present. Comments present and reflection questions are fully answered and variable prefixing and initial suffixing.	but answered vaguely or copied or Variable prefixing/suffixing is present	and <u>No</u> variable prefixing/suffixing.
--	---	---	---